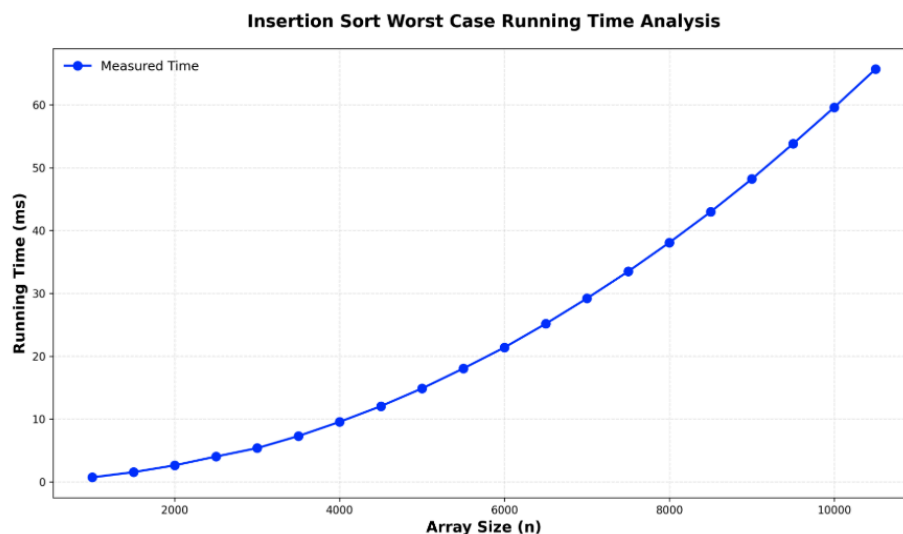


Assignment 4: LargestNumberSolver

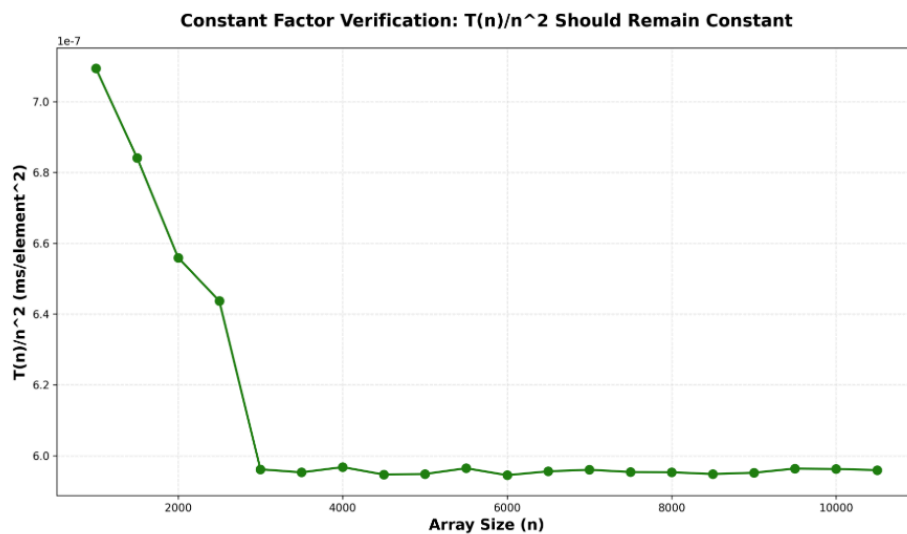
1. My programming partner is Zewei Ren and I submitted the program to Gradescope.
2.
 - 2.1. My programming partner and I switch roles about once an hour. I prefer to switch less often because I believe programming requires maintaining a consistent thought process, and switching too frequently makes it difficult for us to focus on the project.
 - 2.2. We applied pair programming techniques effectively in this assignment; I used these techniques to write about 60% of the code.
 - 2.3. I plan to spend more time thinking about edge testing during my next pair programming session.
 - 2.4. It took me about 6 hours to complete the programming and testing parts of this assignment.
 - 2.5. I felt that my programming partner and I communicated very frequently during the programming process, and I think he had a very positive attitude towards the assignment. I will continue to collaborate with him.
3. In Java, 8 bytes of memory (i.e., $8 \times 8 = 64$ bits) are used to store a long type data. Each bit (short for binary digit) represents either 0 or 1. This means that 64 bits can be used to represent $2^{64} = 18,446,744,073,709,551,616$ different things. For integers, half of them, or 9,223,372,036,854,775,808, are negative integers, and the other half are non-negative integers. Therefore, 9,223,372,036,854,775,807 is the largest integer that can be represented using the integer data type.
In Java, 10 bytes of memory (i.e., $10 \times 8 = 80$ bits) are used to store a long type data. Each bit (short for binary digit) represents either 0 or 1. This means that 80 bits can be used to represent 2^{80} equal numbers. 1,208,925,819,614,629,174,706,176 There are two distinct things. For integers, half of them, or 604,462,909,807,314,587,353,088, are negative integers, and the other half are non-negative integers. Therefore, 604,462,909,807,314,587,353,087 is the largest integer that can be represented using the integer data type.
4.
 - 4.1. insertionSortThe problem size of the method is n , which is the length of the array to be sorted.

- 4.2. The worst-case behavior of the insertion sort method is: $O(n^2)$ - Quadratic time complexity. The worst-case scenario for the sorting algorithm is when the original array is sorted in the opposite way to the desired sorting: the first element is moved 0 times, the second element is moved 1 time, the third element is moved 2 times, and the n th (last) element is moved $(n-1)$ times, for a total of $0+1+2+\dots+(n-1)=n(n-1)/2$ times, denoted in Big O notation as... $O(n^2)$ - Quadratic time complexity
- 4.3. I created an InsertionSortWorstCaseTimingExperiment class that tests insertion sort under worst-case conditions. The experiment generates descending arrays using `ArrayGenerator.generateDescendingArray()` and sorts them with a descending comparator. This combination forces every element to be compared and shifted maximally, creating worst-case behavior. The experiment tests array sizes from 1000 to 10500 elements (incrementing by 500), with 30 iterations per size to ensure reliable averages. Each iteration regenerates a fresh descending array in `runComputation()` to guarantee authentic worst-case conditions on every test.



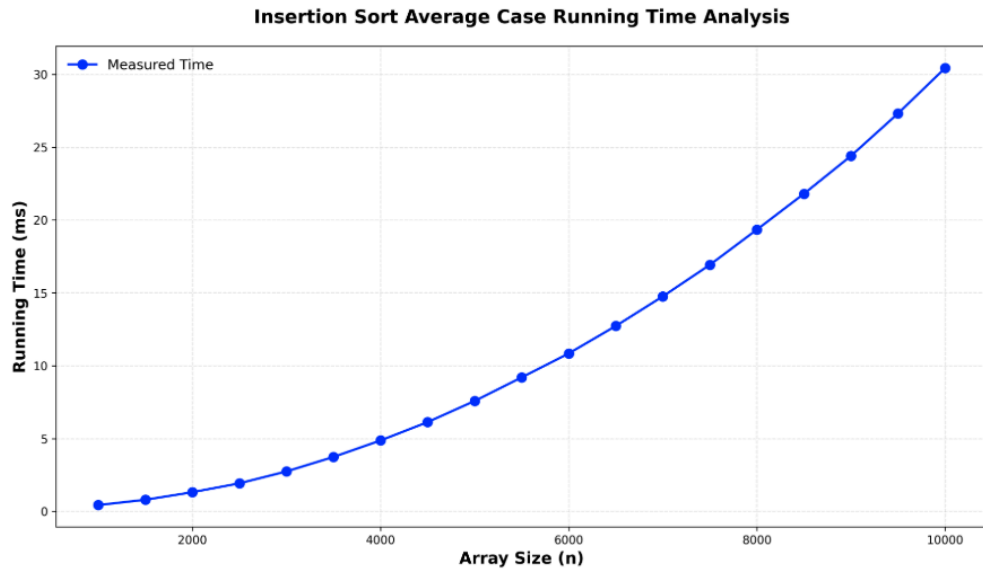
- 4.4.
- 4.5. The plotted runtime growth rate matches my predicted Big O behavior. The value of $T(n)/n^2$ approaches a constant value as the problem size

increases.



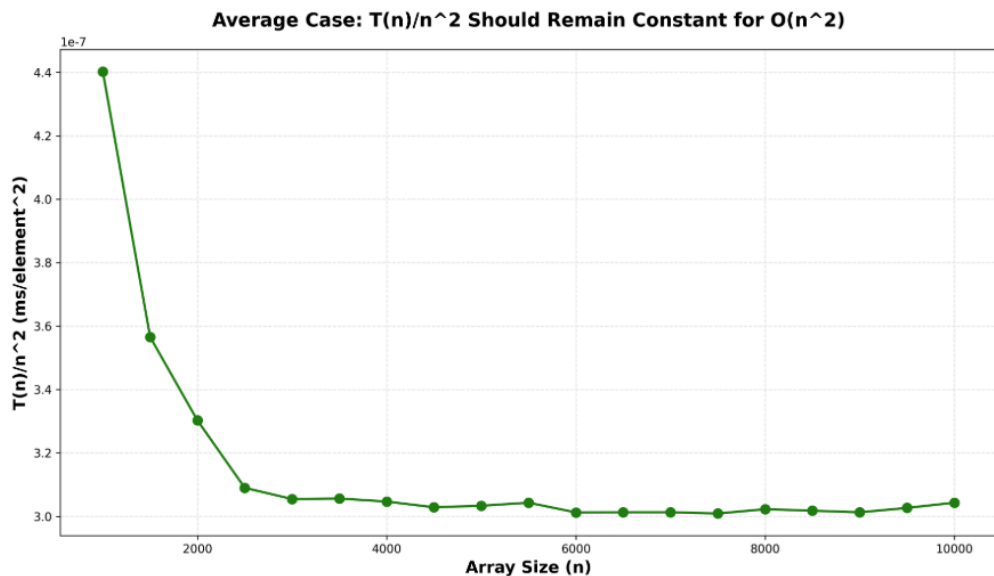
5.

- 5.1. insertionSortThe problem size of the method is n , which is the length of the array to be sorted.
- 5.2. The behavior of the insertion sort method under average conditions is $O(n^2)$ - Quadratic time complexity. Compared to the worst case, each unsorted element needs to be compared with half of the sorted elements above it, which means completing half of the worst-case operations, for a total of $n(n-2)/4$ comparisons, denoted in Big O notation as... $O(n^2)$ - Quadratic time complexity
- 5.3. I created an InsertionSortAverageCaseTimingExperiment class that tests insertion sort under average-case conditions. The experiment generates randomly permuted arrays using `ArrayGenerator.generatePermutedArray()` and sorts them with an ascending comparator. Random permutations require varying amounts of element movement, representing typical real-world scenarios. The experiment tests array sizes from 1000 to 10000 elements (step 500), with 50 iterations per size. Each iteration regenerates a fresh random permutation to ensure representative average-case behavior.



5.4.

5.5. The plotted runtime growth rate matches my predicted Big O behavior. The value of $T(n)/n^2$ approaches a constant value as the problem size increases.



6.

6.1. insertionSortThe problem size of the method is n , which is the length of the array to be sorted.

6.2. The best-case behavior of the insertion sort method is: $O(n)$ - Linear time complexity. The best-case scenario is that the original list is already sorted.insertionSortThe method only needs to traverse $(n-1)$ elements without insertion, and takes a total of $n-1$ iterations, denoted in Big O notation as... $O(n)$ - Linear time complexity

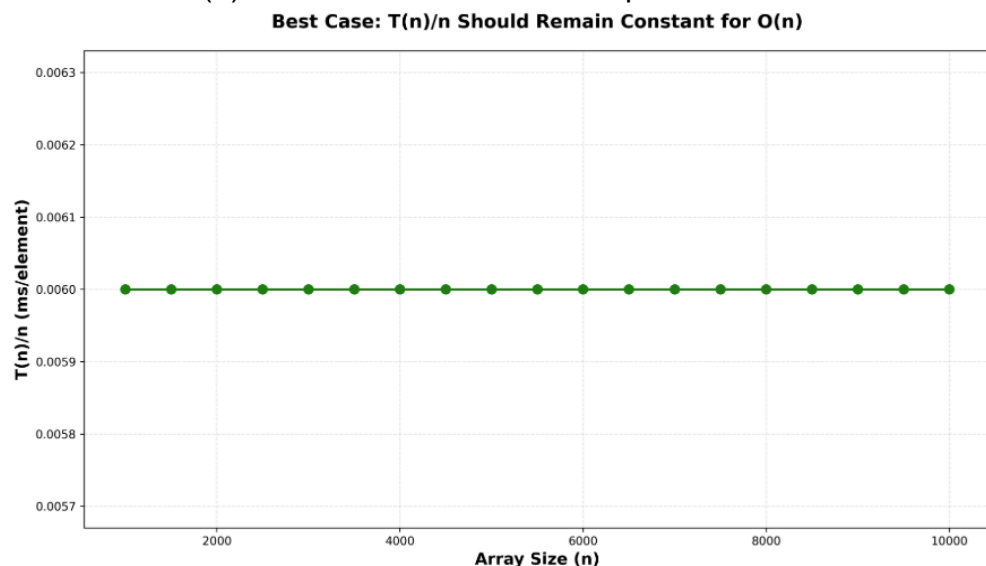
6.3. I created an InsertionSortBestCaseTimingExperiment class that tests insertion sort under best-case conditions. The experiment generates nearly-ascending arrays using

ArrayGenerator.generateNearlyAscendingArray() and sorts them with a descending comparator. This pairing creates best-case conditions because the array's natural order matches the comparator's expected order, requiring minimal comparisons and no element shifting. The experiment tests array sizes from 1000 to 10000 elements (step 500), with 50 iterations per size, regenerating arrays in each iteration to maintain consistent best-case conditions.



6.4.

6.5. The plotted runtime growth rate matches my predicted Big O behavior. The value of $T(n)/n$ remains constant as the problem size increases.



6.6.

7. Assuming there are m arrays as input, each with n elements.

7.1. The problem size for the insertion sort that orders the indices by largest numbers is m . This insertion sort operates on the indices array, which has length m , to determine which array contains the k -th largest number.

- 7.2. The problem size of insertion sort, which generates the largest number each time, is the number of elements in the array currently being sorted(n). This is because the object of insertion sort is the data elements contained in the current array.
- 7.3. I believe that insertion sort, which generates the largest number each time, is dominant.
The overall running time is calculated because this algorithm runs once for each array element, while the other algorithm runs only once. The time complexity of insertion sort mentioned in 7.1 is $O(m^2)$, while the time complexity of insertion sort mentioned in 7.2 is $O(m \cdot n^2)$. Clearly, insertion sort, which generates the largest number each time, dominates.
Overall runtime.